EXPERIMENT 4

TITLE : Variational Autoencoder (VAE) for Image Generation, Reconstruction and Latent Space Visualization

AIM : To implement a Variational Autoencoder (VAE) using TensorFlow/Keras for image reconstruction and generation, visualize the latent space, and understand the role of encoder, decoder and latent variables in Generative AI.

THEORY :

A **Variational Autoencoder (VAE)** is a type of generative deep learning model used for learning meaningful representations of data, reconstructing input data, and generating new samples. It is an extension of the traditional Autoencoder, but instead of encoding an input into a single fixed vector, a VAE represents the input as a probability distribution in a continuous latent space. This makes VAEs useful in Generative AI applications such as image generation, data augmentation, anomaly detection, and representation learning.

A VAE consists of two main components: the Encoder and the Decoder. The encoder takes an input image and compresses it into a lower-dimensional latent representation. Instead of producing a single latent vector, the encoder generates two parameters: the mean (μ) and variance (σ^2) of the latent distribution. A latent vector z is then sampled from this distribution using the reparameterization trick:

$$\mathbf{z = \mu + \sigma \times \varepsilon}$$

where ε is a random value sampled from a standard normal distribution. The reparameterization trick allows the sampling process to remain differentiable so that the model can be trained using backpropagation.

The Decoder receives the sampled latent vector and attempts to reconstruct the original input image. During training, the VAE minimizes a combined loss function consisting of Reconstruction Loss and Kullback-Leibler (KL) Divergence Loss.

Total Loss = Reconstruction Loss + KL Divergence Loss

The Reconstruction Loss measures the difference between the original image and the reconstructed image. A lower reconstruction loss indicates that the decoder is able to reproduce the input more accurately. The KL Divergence Loss ensures that the learned latent distribution remains close to a standard normal distribution. This creates a smooth and continuous latent space from which new samples can be generated.

In this experiment, the MNIST handwritten digit dataset is used. MNIST contains grayscale images of handwritten digits from 0 to 9, with each image having a size of 28 × 28 pixels. The images are normalized and provided to the encoder. After training, the decoder can reconstruct the input images and can also generate new handwritten digit-like images by providing randomly sampled latent vectors.

If the latent dimension is set to two, the learned latent representations can be plotted on a 2D graph. This is called latent space visualization. Images belonging to similar digit classes generally appear close to each other, forming clusters. This visualization helps in understanding how the VAE organizes and represents the features of different images.

Thus, a VAE learns both a compact representation of the input data and the underlying probability distribution of that data. Its ability to reconstruct existing images and generate new images makes it an important model for understanding Generative AI and deep learning.

MNIST DATASET : https://www.kaggle.com/datasets/hojjatk/mnist-dataset

COLAB FILE: **GENAI EXP4 HARSHAL/RAJ/07/21.ipynb**

CODE AND OUTPUT :

Step 1 & 2: Import Required Libraries

```
!pip install -q tensorflow matplotlib numpy scikit-learn

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.manifold import TSNE

print("TensorFlow version:", tf.__version__)
tf.random.set_seed(42)
np.random.seed(42)
```

```
TensorFlow version: 2.20.0
```

Step 3 & 4: Load the MNIST Dataset and Normalize Pixel Values

The MNIST dataset is loaded directly from `tensorflow.keras.datasets`, so **no manual dataset download/link is required**. Pixel values are scaled from the range [0, 255] to [0, 1].

```
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

# Normalize pixel values to [0, 1]
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Flatten 28x28 images into 784-dim vectors
image_size = x_train.shape[1] * x_train.shape[2]
x_train_flat = x_train.reshape((-1, image_size))
x_test_flat = x_test.reshape((-1, image_size))

print("Train shape:", x_train_flat.shape)
print("Test shape :", x_test_flat.shape)

# Show a few sample digits
fig, axes = plt.subplots(1, 8, figsize=(12, 2))
for i, ax in enumerate(axes):
    ax.imshow(x_train[i], cmap="gray")
    ax.axis("off")
plt.suptitle("Sample MNIST Digits")
plt.show()
```

```
Train shape: (60000, 784)
Test shape : (10000, 784)
```

Sample MNIST Digits



Step 5 & 6: Encoder Network and Reparameterization (Sampling) Layer

The encoder compresses each input image into a latent mean (`z_mean`) and log-variance (`z_log_var`). The **Reparameterization Trick** samples a latent vector `z = mean + sigma * epsilon`, where `epsilon ~ N(0, 1)`, keeping the sampling step differentiable so gradients can flow during backpropagation.

```
[ ]
latent_dim = 2  # 2D latent space, chosen so it can be visualized directly on a scatter plot

# ---- Encoder ----
encoder_inputs = keras.Input(shape=(image_size,), name="encoder_input")
x = layers.Dense(256, activation="relu")(encoder_inputs)
x = layers.Dense(64, activation="relu")(x)

z_mean = layers.Dense(latent_dim, name="z_mean")(x)
z_log_var = layers.Dense(latent_dim, name="z_log_var")(x)

class Sampling(layers.Layer):
    """Reparameterization trick: z = mean + std * epsilon"""
    def call(self, inputs):
        z_mean, z_log_var = inputs
        batch = tf.shape(z_mean)[0]
        dim = tf.shape(z_mean)[1]
        epsilon = tf.random.normal(shape=(batch, dim))
        return z_mean + tf.exp(0.5 * z_log_var) * epsilon

z = Sampling(name="z")([z_mean, z_log_var])

encoder = keras.Model(encoder_inputs, [z_mean, z_log_var, z], name="encoder")
encoder.summary()
```

```
Model: "encoder"
```

Layer (type)	Output Shape	Param #	Connected to
encoder_input (InputLayer)	(None, 784)	0	-
dense_10 (Dense)	(None, 256)	200,960	encoder_input[0]…
dense_11 (Dense)	(None, 64)	16,448	dense_10[0][0]
z_mean (Dense)	(None, 2)	130	dense_11[0][0]
z_log_var (Dense)	(None, 2)	130	dense_11[0][0]
z (Sampling)	(None, 2)	0	z_mean[0][0], z_log_var[0][0]

```
Total params: 217,668 (850.27 KB)
Trainable params: 217,668 (850.27 KB)
Non-trainable params: 0 (0.00 B)
```

Step 7: Decoder Network

+ Code + Text

```
# ---- Decoder ----
latent_inputs = keras.Input(shape=(latent_dim,), name="decoder_input")
x = layers.Dense(64, activation="relu")(latent_inputs)
x = layers.Dense(256, activation="relu")(x)
decoder_outputs = layers.Dense(image_size, activation="sigmoid")(x)

decoder = keras.Model(latent_inputs, decoder_outputs, name="decoder")
decoder.summary()
```

```
Model: "decoder"
```

Layer (type)	Output Shape	Param #
decoder_input (InputLayer)	(None, 2)	0
dense_12 (Dense)	(None, 64)	192
dense_13 (Dense)	(None, 256)	16,640
dense_14 (Dense)	(None, 784)	201,488

```
Total params: 218,320 (852.81 KB)
Trainable params: 218,320 (852.81 KB)
Non-trainable params: 0 (0.00 B)
```

Step 8: Define the Full VAE Model and Loss Function

```
class VAE(keras.Model):
    def __init__(self, encoder, decoder, **kwargs):
        super().__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder
        self.total_loss_tracker = keras.metrics.Mean(name="total_loss")
        self.reconstruction_loss_tracker = keras.metrics.Mean(name="reconstruction_loss")
        self.kl_loss_tracker = keras.metrics.Mean(name="kl_loss")

    @property
    def metrics(self):
        return [self.total_loss_tracker, self.reconstruction_loss_tracker, self.kl_loss_tracker]

    def train_step(self, data):
        if isinstance(data, tuple):
            data = data[0]
        with tf.GradientTape() as tape:
            z_mean, z_log_var, z = self.encoder(data)
            reconstruction = self.decoder(z)

            reconstruction_loss = tf.reduce_mean(
                tf.reduce_sum(keras.losses.binary_crossentropy(data, reconstruction), axis=-1)
            )

            kl_loss = -0.5 * (1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var))
            kl_loss = tf.reduce_mean(tf.reduce_sum(kl_loss, axis=1))

            total_loss = reconstruction_loss + kl_loss

        grads = tape.gradient(total_loss, self.trainable_weights)
        self.optimizer.apply_gradients(zip(grads, self.trainable_weights))

        self.total_loss_tracker.update_state(total_loss)
        self.reconstruction_loss_tracker.update_state(reconstruction_loss)
        self.kl_loss_tracker.update_state(kl_loss)
        return {
            "loss": self.total_loss_tracker.result(),
```

Step 9: Train the VAE (20–30 Epochs)

```
EPOCHS = 30
BATCH_SIZE = 128

print(f"Training VAE for {EPOCHS} epochs with batch size {BATCH_SIZE}...")
history = vae.fit(x_train_flat, epochs=EPOCHS, batch_size=BATCH_SIZE, verbose=0) # Set verbose to 0 for cleaner output
print("VAE training complete.")
```

```
Training VAE for 30 epochs with batch size 128...
VAE training complete.
```

Training Loss Graph

```
plt.figure(figsize=(8, 5))
plt.plot(history.history["loss"], label="Total Loss")
plt.plot(history.history["reconstruction_loss"], label="Reconstruction Loss")
plt.plot(history.history["kl_loss"], label="KL Divergence Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("VAE Training Loss Curve")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

Step 10: Reconstruct Test Images and Compare with Originals

```
n = 10  # number of digits to display
z_mean_test, z_log_var_test, z_test = encoder.predict(x_test_flat[:n])
reconstructed = decoder.predict(z_test)

plt.figure(figsize=(20, 4))
for i in range(n):
    # Original
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test_flat[i].reshape(28, 28), cmap="gray")
    plt.title("Original")
    plt.axis("off")

    # Reconstructed
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(reconstructed[i].reshape(28, 28), cmap="gray")
    plt.title("Reconstructed")
    plt.axis("off")

plt.suptitle("Original vs Reconstructed Images")
plt.show()
```

Step 11: Generate New Images by Sampling Random Latent Vectors

```
n = 10
random_latent_vectors = np.random.normal(size=(n, latent_dim))
generated_images = decoder.predict(random_latent_vectors)

plt.figure(figsize=(20, 2))
for i in range(n):
    ax = plt.subplot(1, n, i + 1)
    plt.imshow(generated_images[i].reshape(28, 28), cmap="gray")
    plt.axis("off")
plt.suptitle("Newly Generated Digits from Random Latent Vectors")
plt.show()
```

Step 12: Visualize the 2-D Latent Space

```
z_mean_all, _, _ = encoder.predict(x_test_flat)

plt.figure(figsize=(10, 8))
scatter = plt.scatter(z_mean_all[:, 0], z_mean_all[:, 1], c=y_test, cmap="tab10", s=5, alpha=0.7)
plt.colorbar(scatter, label="Digit Label")
plt.xlabel("Latent Dimension 1")
plt.ylabel("Latent Dimension 2")
plt.title("Latent Space Visualization (Encoded Test Samples)")
plt.grid(alpha=0.3)
plt.show()
```

CONCLUSION :

The Variational Autoencoder was successfully studied as a generative deep learning model consisting of an encoder, latent space, and decoder. The encoder learns the mean and variance of a probability distribution from the input image, while the reparameterization trick enables efficient gradient-based training by allowing a latent vector to be sampled in a differentiable manner.

The VAE uses a combination of reconstruction loss and KL divergence loss to learn both accurate image reconstruction and a smooth, meaningful latent space. Using the MNIST dataset, the model can reconstruct handwritten digits and generate new digit-like images by sampling random points from the latent space.

The visualization of the latent space demonstrates how similar images are grouped together and provides an intuitive understanding of how neural networks learn compressed representations. Overall, this experiment provides a practical understanding of the working principles of Variational Autoencoders and their importance in Generative AI, particularly in image generation, reconstruction, anomaly detection, data augmentation, and representation learning.